

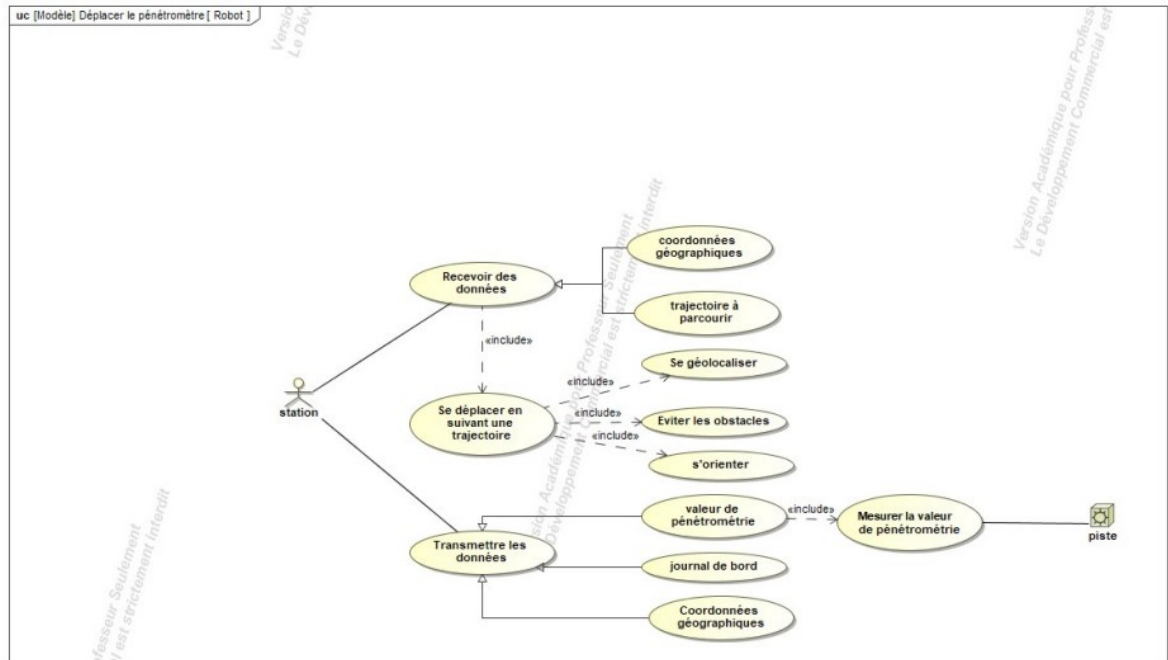
Table des matières

1 -SITUATION DANS LE PROJET	2
1.1 -SYNOPSIS DE LA REALISATION	2
1.2 -DESCRIPTION DE LA PARTIE PERSONNELLE	2
2 -REALISATION DU CAS D'UTILISATION FS51 : RECEVOIR DES DONNEES	3
2.1 -CONCEPTION DETAILLEE	3
2.1.1 -Objectif fonctionnel	3
2.1.2 -Architecture de communication	3
2.1.3 -Programmation du module NRF9151	3
2.1.4 -Setup de l'environnement de développement	4
2.1.5 -Liaison UART avec l'ESP32-S2	5
2.1.6 -Contraintes techniques rencontrées	5
2.1.7 -Résultat obtenu	5
2.2 -TESTS UNITAIRES	5
2.2.1 -Test unitaire du SOC NRF9151	5
2.2.2 -Problèmes rencontrés	6
3 -REALISATION DU CAS D'UTILISATION FS53 : SE GEOLOCALISER.....	6
3.1 -CONCEPTION DETAILLEE	6
3.1.1 -Objectif fonctionnel	6
3.1.2 -Module utilisé	6
3.1.3 -Traitement des données GPS	6
4 -REALISATION DU CAS D'UTILISATION FS54 : S'ORIENTER	6
4.1 -CONCEPTION DETAILLEE	6
4.1.1 -Objectif fonctionnel	7
4.1.2 -Calcul de l'angle d'orientation	7
4.1.3 -Ajustement de la direction	7
5 -REALISATION DU CAS D'UTILISATION FS55 : TRANSMETTRE DES DONNEES	7
5.1 -CONCEPTION DETAILLEE	7
5.1.1 -Objectif fonctionnel	7
5.1.2 -Protocole de transmission	7
5.2 -TESTS UNITAIRES	7
5.2.1 -Test unitaire du SIM7080G	7
5.2.2 -Problèmes rencontrés	8
6 -BILAN DE LA REALISATION PERSONNELLE.....	8
6.1 -TACHES REALISEES	8
6.2 -COMPETENCES MOBILISEES	8
6.3 -DIFFICULTES RENCONTREES	9
6.4 -PISTES D'AMELIORATION	9

1 - Situation dans le projet

1.1 - Synoptique de la réalisation

Ce projet mise à entretenir en surveillant l'état du terrain de course d'un hippodrome grâce à des relevés météorologique ainsi que des données d'humidité et de Pénétrométrie du sol de manière semi-autonome (les relevés sont réalisés sans intervention d'un operateur mais analyser par celui-ci pour maintenir l'entretien).



Fonctions de services du (Fxxx)	Description
Fs51	Recevoir des données
Fs52	Se déplacer en suivant une trajectoire. La trajectoire est établie à partir des coordonnées et de l'orientation du robot,
Fs53	Se géolocaliser
Fs54	S'orienter
Fs55	Transmettre des données
Fs56	Mesurer la valeur de pénétrométrie
Fs57	Éviter les obstacles ; Des capteurs tel que palpeurs, détecteur de vide, gyroscope et lidar peuvent être envisagés.

Liste des fonctions assurées par l'étudiant : Fs5 : Déplacer le pénétromètre FS51 : Recevoir des données FS53 : Se géolocaliser ; FS54 : S'orienter ; FS55 : Transmettre des données	Installation : - Robot équipé du pénétromètre. Mise en œuvre : - Robot et pénétromètre (dans un second temps). Réalisation : - carte permettant de communiquer, de géolocaliser et s'orienter Documentation : - Installation - Conception détaillée - Mise en service et exploitation
--	---

1.2 - Description de la partie personnelle

- Mon travail est de faire en sorte que le robot reçoive une direction à prendre ainsi qu'une distance à parcourir pour garantir qu'il se déplace au bon endroit. Ainsi que sur le robot déterminé son orientation ainsi que sa position.

- Le projet a été codé dans son intégralité sur vscode pour se faire j'ai dû mettre l'extension esp-idf pour utiliser l'esp32 avec vscode et pour l'utilisation du nrf9151 j'ai suivi la procédure fournie lorsqu'on lance le logiciel nrfconnect se qui me redirige vers vscode. La réalisation du PCB a quand à elle été réalisée avec easyEDA pour sa grande bibliothèque d'empreintes.
- Lors du projet j'ai choisi un esp32-s3 pour c'est performance sachant que le prix était très proche d'un esp32 classique j'ai préféré opter pour cette solution. J'ai opté pour un esp32 car simple à prendre en main et peut cher. Ensuite au niveau du GPS je n'ai pas trouvé beaucoup de module correspondant à mon besoin, il me fallait un GPS d'une précision de 1m et c'est le moins cher que j'ai pu trouver et qui correspondait à m'est contrainte.
- Ma partie se situe au niveau de la communication pour le control du robot. Le robot est contrôlé depuis une station externe au robot qui doit envoyer toutes les informations via mon module LTE-M NRF9151 que j'ai choisi car peut cher et programmable ce qui me permet de dissocier l'orientation du robot ainsi que la communication de l'avancement.
- ESP-IDF : <https://docs.espressif.com/projects/esp-idf/en/v4.3.1/esp32/get-started/vscode-setup.html>
- Parameter LITTLEFS : <https://docs.espressif.com/projects/vscode-esp-idf-extension/en/latest/additionalfeatures/partition-table-editor.html>
- NRF Connect : <https://www.nordicsemi.com/Products/Development-tools/nRF-Connect-for-VS-Code#:~:text=Nordic%20Semiconductor's%20nRF%20Connect%20extension%20pack%20turns%20VS,nRF52%20Series%20devices%20on%20Windows%2C%20macOS%20or%20Linux.>

2 - Réalisation du cas d'utilisation FS51 : Recevoir des données

2.1 - Conception détaillée

Le cas d'utilisation **FS51 – Recevoir des données** correspond à la capacité du robot à récupérer des instructions de navigation envoyées par un serveur distant. Ces données sont ensuite utilisées pour piloter les moteurs et ajuster l'orientation du robot de manière autonome.

2.1.1 - Objectif fonctionnel

L'objectif est de permettre au robot de recevoir, via un réseau cellulaire bas débit (LTE-M), trois types d'informations :

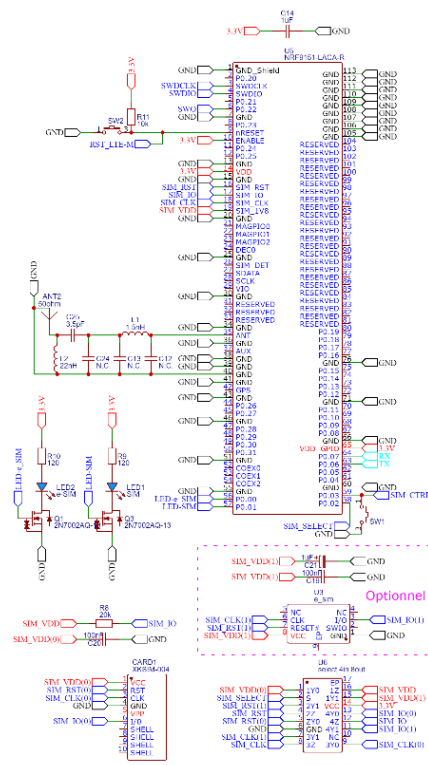
- Une **distance à parcourir** (exprimée en centimètres),
- Une **orientation à adopter** (exprimée en degrés ou en direction relative).
- Un état booléen sur si c'est un point de pénétrométrie ou non

Ces données sont transmises par le serveur à un module embarqué sur le robot, puis exploitées localement pour assurer un déplacement autonome.

2.1.2 - Architecture de communication

Pour répondre à cet objectif, l'architecture suivante a été mise en place :

- Utilisation d'un **module Nordic Semiconductor NRF9151**, intégrant :
 - Un microcontrôleur **ARM Cortex-M33**,
 - Un **modem LTE-M**,
 - Une gestion possible de carte **eSIM**,
 - Une **faible consommation électrique**.
- Le **NRF9151** reçoit les données du serveur via le réseau cellulaire, les decode, puis les transmet via une liaison **UART** au **microcontrôleur ESP32-S2**



Module LTE-M

2.1.3 - Programmation du module NRF9151

Le module NRF9151 est livré non programmer. Il a donc fallu :

- Le **flasher** via son interface **SWD** (Serial Wire Debug),
- Développer un **firmware personnalisé** pour assurer la gestion de la communication cellulaire et de la liaison UART.

🔧 Programmeur utilisé

Un **ST-LINK V2** a été utilisé, converti en **J-LINK compatible** grâce à un utilitaire officiel fourni par **Segger**, permettant :

- La connexion au port SWD du NRF9151,
- Le flashage et le débogage via l'environnement Nordic.

Environnement de développement

Le développement a été réalisé avec :

- **Visual Studio Code** (IDE),
- **nrfutil** pour la compilation et le flashage,
- **Nordic SDK** (nRF Connect SDK).

2.1.4 - Setup de l'environnement de développement

Cette section explique **comment reproduire mon environnement de développement** afin de programmer et flasher le module **nRF9151** dans les mêmes conditions que celles utilisées dans ce projet.

1. Prérequis matériels

- Un ordinateur sous **Windows 10/11** ou **Linux** (Ubuntu recommandé),
- Un programmeur **J-LINK** (ou **ST-LINK** avec étape2),
- Un câble micro-USB vers USB,
- Le module **nRF9151** soudé sur une carte de test ou un PCB personnel,
- Une connexion Internet.

2. (Optionnel) Conversion ST-LINK en J-LINK (Segger)

1. Télécharger l'outil **J-Link ST-Link Reflash Utility** depuis le site officiel Segger :
<https://www.segger.com/downloads/jlink>
- 1.bis Si vous utilisez un ST-Link non officiel, essayez d'utiliser :
https://signetik.zendesk.com/hc/en-us/article_attachments/4407367401751/3_STLinkReflash_Modified.zip
2. Lancer l'outil, connecter le ST-LINK via USB, puis suivre les étapes de conversion.
3. Une fois terminé, le ST-LINK est reconnu comme **J-LINK OB** dans le gestionnaire de périphériques.

3. Installation de Visual Studio Code + outils Nordic

1. Télécharger et installer **Visual Studio Code** :
<https://code.visualstudio.com>
2. Installer l'extension **nRF Connect for VS Code** depuis le Marketplace.
3. Lancer **nRF Connect for Desktop** (téléchargeable ici :
<https://www.nordicsemi.com/Products/Development-tools/nRF-Connect-for-desktop>)
4. Installer les modules suivants :
 - **Toolchain Manager**
 - **nRF Connect SDK (v2.x recommandé)**
(par exemple v2.4.2)
5. Installer nRFUtil depuis : <https://www.nordicsemi.com/Products/Development-tools/nRF-Util>
Une fois installé, placer le à la racine de votre système et ajouter son chemin aux variables d'environnement.
Pour vérifier si cela est correctement installé, il suffit de taper nrfutil dans la console Windows.
6. Dans Toolchain Manager, cliquer sur "Open VS Code" à côté de l'environnement SDK.

4. Configuration du projet firmware

1. Créer un nouveau projet Zephyr ou cloner un projet d'exemple Nordic.
Exemple de chemin :
nrf/samples/nrf9160/mqtt_simple (pour tester LTE-M)
2. Adapter les fichiers :
 - prj.conf pour activer LTE, UART, logs, etc.
 - overlay.conf pour la gestion de pins et GPIO.
3. Compiler le projet en faisant ([CTRL] + [ALT] + N) puis en cliquant sur build
4. Flasher le module en cliquant sur flash une fois le build effectué

5. Vérification

- Utiliser **nRF Terminal** (intégré à VS Code) ou un terminal série comme **Tera Term** pour observer les logs via UART.
- Vérifier la bonne connexion au réseau (LTE), et la réponse du modem grâce au cellular monitor de nRF Connect.

2.1.5 -Liaison UART avec l'ESP32-S2

Une fois les données reçues, le NRF9151 les transmet à l'ESP32-S2 via une liaison série **UART**. Le protocole mis en place utilise :

- Une **structure de trame simple** : [Start Byte] – [Type de commande] – [Données]– [End Byte],



Traitement par l'ESP32-S2 :

- Analyse de la trame reçue,
- Interprétation des données (distance et orientation)

2.1.6 -Contraintes techniques rencontrées

Contraintes	Solutions apportées
Coût élevé du programmeur J-LINK	Conversion d'un ST-LINK V2 en J-LINK
Faible documentation du NRF9151	Étude des ressources Nordic + firmware personnalisé
Besoin d'un système basse consommation	Choix de composants adaptés + gestion des modes veille

2.1.7 -Résultat obtenu

Le système permet désormais au robot :

- De recevoir efficacement des commandes à distance via LTE-M,
- De transférer ces commandes à son unité centrale

Ce fonctionnement est la base de la navigation autonome du robot.

2.2 - Tests unitaires

2.2.1 -Test unitaire du SOC NRF9151

Identification du Test		LTE-CLIENT1		Référence du Module Testé:	NRF9151	
Objectif du test :		Test connexion TCP via LTE-M en tant que client				
Nom du testeur :		GENEVAISE		Date :	02/05/2025	
Moyens mis en œuvre :		Logiciel : Terminal port série	Matériel : PC	Outils de développement : /		
Prérequis/Préconditions		Programme flasher dans le microcontroller Server TCP actif et Tester				
Num étape	Procédure du test :	Résultat attendu		Critères de Réussite	Résultat obtenu	Validation (O/N)
1	Brancher le module au pc puis ouvrir le port série. Ensuite alimenté le module et attendre "RRC mode:" sur le port série.	Affichage de "RRC mode:" sur le port série		La LED1 clignotte de manière régulière. Et le mode RRC est Idle ou Connected	RRC mode: Idle	O
2	Appuye sur le bouton1	Le port série du nrf9151 retourne "Message sent successfully." et on récupère "Hello from nRF9151 Client!" sur le server "TCP"		le server TCP reçoit le message attendu après que le bouton 1 est été presser sur le module	"Hello from nRF9151 Client!" sur le server "TCP"	O

3	Attendre de recevoir une réponse du server après "Message sent successfully."	Le port série retourne "Received response from server:...."	Réception d'un message du server	Le server retourne un message et le message est correctement reçu.	O
4	Attendre de recevoir le message de fermeture de la connexion	La connexion est fermée(vérifié sur le server pour être sûr)	La connexion est fermée	"Socket closed."	O
Résultats	[X] OK Anomalie [] mineure [] majeure [] bloquante [] NOK			Réf. anomalie :	
Conclusion du test :		Communication LTE-M du NRF9151 fonctionnelle			

2.2.2 - Problèmes rencontrés

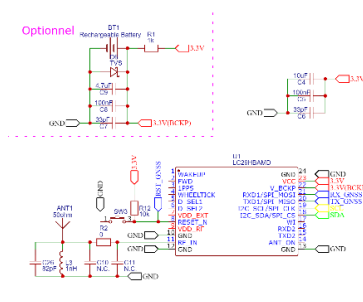
Nécessite une interface SWD pour flasher le programme (J-LINK recommander)

Solution :

- Utiliser un ST-LINK (Peut onéreux) et le convertir en J-LINK grâce à un logiciel fourni par SEGGER (OFFICIAL)

3 - Réalisation du cas d'utilisation FS53 : Se géolocaliser

3.1 - Conception détaillée



Module GNSS

Le cas d'utilisation **FS53 – Se géolocaliser** permet au robot d'obtenir ses coordonnées géographiques en temps réel via un module GPS. Cette localisation est indispensable pour déterminer la position du robot, s'orienter correctement, et valider sa présence dans la zone prévue par la station.

3.1.1 - Objectif fonctionnel

L'objectif est de permettre au robot de :

- Déterminer sa position **latitude/longitude** en temps réel,
- Utiliser cette position pour **authentification** et **guidage** vers un objectif.

3.1.2 - Module utilisé

Le module GPS utilisé est le **LC29HBAMD**, choisi pour :

- Sa **haute précision** en espace dégagé,
- Sa **consommation faible**, adaptée à l'embarqué,
- Sa **communication I²C** simple à interfacer avec l'ESP32-S2,
- Son **format compact** adapté à l'intégration robotique.

3.1.3 - Traitement des données GPS

Contrairement à une liaison série classique, l'ESP32-S2 interroge le GPS via le bus **I²C** en mode esclave pour récupérer les **trames NMEA**.

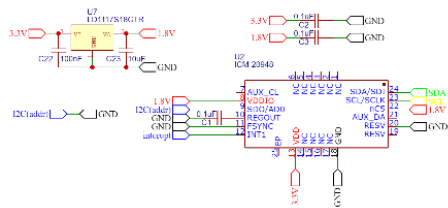
Le fonctionnement mis en place :

- Lecture cyclique (toutes les 200 à 500 ms) d'un buffer interne du GPS,
- Décodage logiciel des trames **NMEA \$GNRMC** et **\$GNGLL**,
- Extraction des données utiles : latitude, longitude.

Cette solution permet de libérer l'UART dans le cas de debug ou de mise à jour du firmware interne et aussi de placer les capteurs sur un bus commun.

4 - Réalisation du cas d'utilisation FS54 : S'orienter

4.1 - Conception détaillée



Magnétomètre

Le cas d'utilisation **FS54 – S'orienter** permet au robot de se tourner dans la bonne direction avant de commencer son déplacement. Cela garantit que la distance parcourue corresponde bien à l'objectif prévu.

4.1.1 - Objectif fonctionnel

Permettre au robot de :

- S'orienter à partir des données d'orientation reçu,
- Adapter sa direction avant déplacement réel.

4.1.2 -Ajustement de la direction

Le robot effectue la rotation grâce à ses moteurs (commande différentielle). Un **capteur inertiel ICM-20948** peut être utilisé pour :

- Suivre l'évolution de l'orientation pendant la rotation,
- Arrêter la rotation dès que l'orientation cible est atteint.

5 - Réalisation du cas d'utilisation FS55 : Transmettre des données

5.1 - Conception détaillée

Le cas d'utilisation **FS55 – Transmettre des données** permet au robot d'envoyer à la station distante les **résultats de mission** : position finale, distance réellement parcourue, et état d'exécution.

5.1.1 -Objectif fonctionnel

Assurer un **retour d'information complet et fiable** vers la station. Ces données sont utilisées pour :

- Vérification que l'objectif a bien été atteint,
- Mise à jour du planning serveur,
- Analyse de l'efficacité du robot.

5.1.2 -Protocole de transmission

L'ESP32-S2 construit une trame structurée contenant :

- L'identifiant du robot,
- Les coordonnées GPS finales,
- La distance parcourue (estimée),
- Le statut de mission : OK, ERREUR, NON FAITE.

La trame est envoyée via **UART** au nRF9151, qui la transmet au serveur distant via **TCP/IP**.

5.2 - Tests unitaires

5.2.1 -Test unitaire du SIM7080G

Identification du Test	LTE-SERVER1			Référence du Module Testé:	Station-Pilote	
Objectif du test :	Test connection TCP via LTE-M en tant que Server					
Nom du testeur :	GENEVAISE			Date :	01/05/2025	
Moyens mis en œuvre :	Logiciel :	Terminal port série Client TCP	Matériel :	PC	Outils de développement :	/
Prérequis/Préconditions	Programme flasher dans le microcontroller connecté au sim7080G					

Num étape	Procédure du test :	Résultat attendu	Critères de Réussite	Résultat obtenu	Validation (O/N)
1	Se connecter via le port série puis appuyer sur le bouton reset de l'esp	Le port série ne retourne aucune erreur	Le programme c'est dérouler correctement	Aucun problème server start	O
2	Se connecter au server via le client TCP	On voit que un client est connecter sur le port série et on reçoit un message de retour sur le client TCP	Le client TCP est connecter et reçoit un message	Client TCP connecter et message reçu	O
Résultats	[X] OK Anomalie [] mineure [] majeure [] bloquante [] NOK			Réf. anomalie :	
Conclusion du test :	Communication LTE-M du SIM7080G fonctionnelle				

5.2.2 - Problèmes rencontrés

Problème de signal

- Solution, relancer le test en plaçant l'antenne dans un lieu dégager.

Problème de réseau introuvable

- Faire un scan réseau pour être sûr qu'il est introuvable
- Sinon juste relancer le test jusqu'à que cela se connecte

6 - Bilan de la réalisation personnelle

Cette partie dresse un bilan technique, méthodologique et personnel des travaux réalisés dans le cadre de ma contribution au projet. Elle met en évidence les compétences mobilisées, les difficultés rencontrées, les solutions apportées, ainsi que les pistes d'amélioration envisagées.

6.1 - Tâches réalisées

Au cours du projet, j'ai pris en charge la conception, le développement et la validation des fonctions suivantes du robot :

- **FS51 – Recevoir des données** : Mise en place de la communication TCP via LTE-M avec un serveur distant, intégration du module **nRF9151**,
- **FS53 – Se géolocaliser** : Intégration du module **LC29HBAMD**, récupération de trames GPS via I²C, extraction des données NMEA,
- **FS54 – S'orienter** : Calcul de l'angle d'orientation à partir des coordonnées GPS, gestion de la rotation du robot vers la bonne direction,
- **FS55 – Transmettre des données** : Création des trames de réponse, transmission des résultats vers la station-pilote via LTE-M,
- **Développement du serveur embarqué** : Mise en place d'un serveur TCP autonome sur **ESP32-S3 + SIM7080G**, configuration de la carte SIM à IP statique.

J'ai également rédigé un firmware embarqué pour le **nRF9151**, et configuré l'environnement de développement Nordic SDK avec **Visual Studio Code** et **nrfutil**.

6.2 - Compétences mobilisées

- **Systèmes embarqués** :
 - Programmation sur **ESP32** (ESP-IDF),
 - Programmation et flashage du **nRF9151** via SWD,
 - Utilisation d'un **capteur GPS** et d'un **capteur inertiel**,
- **Réseaux et télécommunications** :
 - Communication **TCP/IP** sur **réseau cellulaire (LTE-M)**,
 - Paramétrage et exploitation d'une **carte SIM à IP fixe**,
- **Protocoles** :
 - Analyse des trames **NMEA GPS**,

- I²C,
- UART
- **Méthodologie de projet :**
 - Documentation des tests,
 - Suivi des versions,
 - Organisation en modules fonctionnels.

6.3 - Difficultés rencontrées

Difficulté	Solution apportée
Programmation du nRF9151 sans J-LINK	Conversion d'un ST-LINK V2 en J-LINK via utilitaire Segger
Manque de ressources sur le nRF9151	Recherche dans la documentation Nordic et forums développeurs
Instabilité LTE-M en intérieur	Tests en extérieur, changement de zone de couverture
Décodage de trames NMEA via I ² C	Implémentation d'un buffer circulaire côté ESP32-S2
Gestion du timing réveil/planification	Ajout de tolérance de ± 10 s et retour d'état serveur

6.4 - Pistes d'amélioration

Plusieurs améliorations peuvent être envisagées dans une future version du projet :

- Remplacer **LTE-M** par une liaison **Wi-Fi HaLow** (IEEE 802.11ah), permettant :
 - Une portée longue comparable au LTE (jusqu'à 1 km),
 - Une consommation similaire ou inférieure,
 - L'absence de **forfait** ou de **carte SIM**, réduisant les coûts d'exploitation,
 - Une communication locale entre la station et le robot, via un point d'accès dédié.
- Ajout d'un chiffrement (**TLS/SSL**) pour sécuriser les échanges TCP en cas de réseau LTE conservé,
- Liées la station de pilotage du robot à une **IHM**,
- **Optimisation du firmware pour basse consommation**, avec mise en veille profonde des modules non sollicités.